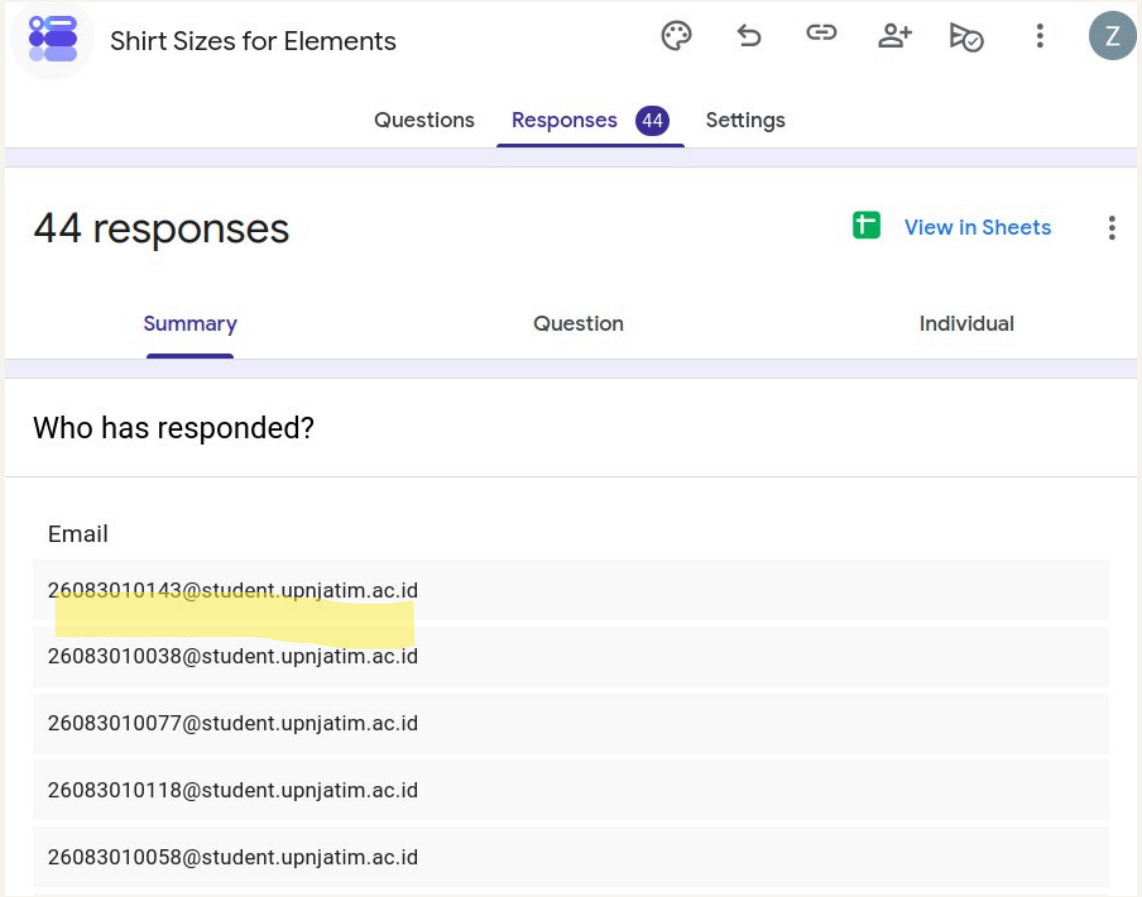
Elements of AI

https://github.com/jendralhxr/introdatascience/blob/main/elemen-bmi.pdf

From raw surveys into clean *engineered* data

Zulhaj Aliansyah

The data



Weight (kg)	Height (cm)
Weight (kg)	Height (cm)
55	165
4 responses	8 responses
45	155
4 responses	5 responses
58	156
4 responses	4 responses

```
import pandas as pd
# Read the raw dataset
df = pd.read_csv('eai-databmi - Form Responses 1.csv')
```

Remove Duplicates

11/3/2026 16:35:58	amadeakhansas@gmail.com	156	51
11/3/2026 16:36:27	amadeakhansas@gmail.com	156	51

```
# Check how many duplicates exist based strictly on 'Email Address'
duplicate_count = df.duplicated(subset=['Email Address']).sum()
print(f"Number of duplicate email entries found: {duplicate_count}")

# View the duplicated entries side-by-side before dropping
duplicate_rows = df[df.duplicated(subset=['Email Address'], keep=False)]
print(duplicate_rows[['Email Address', 'Timestamp', 'Height (cm)', 'Weight
(kg)']])

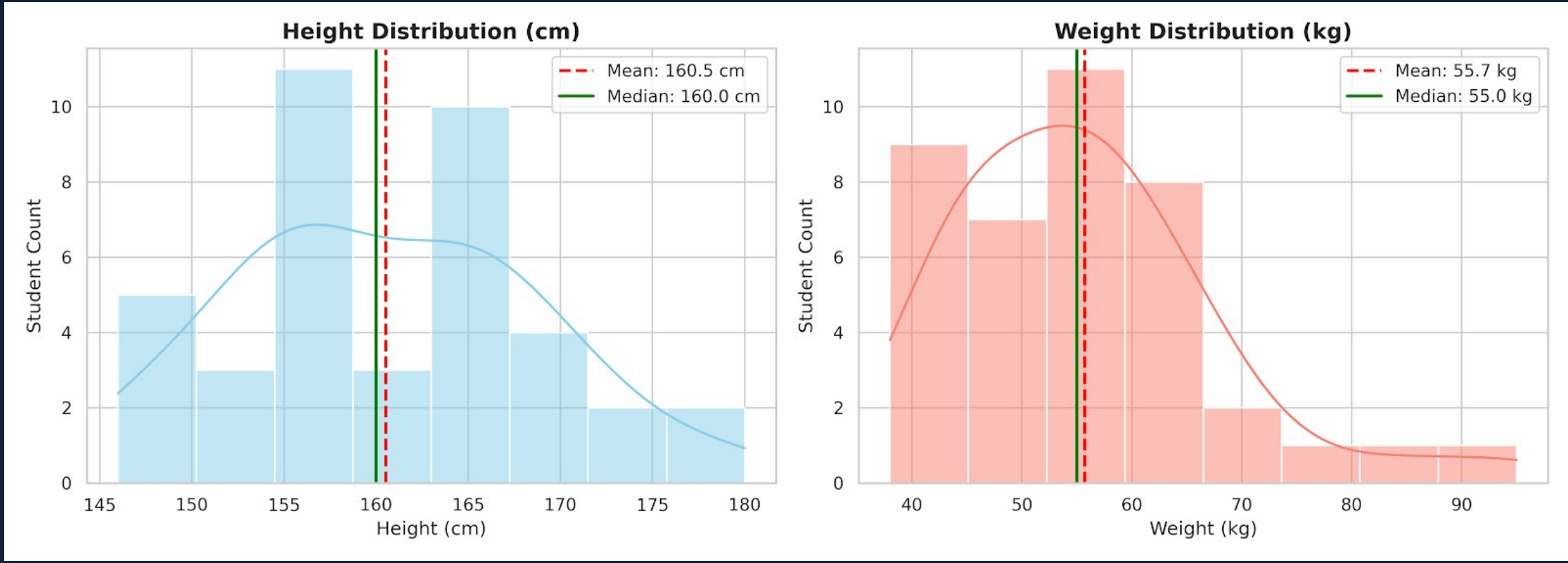
# Remove duplicates based ONLY on 'Email Address'
# keep='last' preserves the latest submission, keep='first' preserves the
earliest
df_cleaned = df.drop_duplicates(subset=['Email Address'],
keep='last').reset_index(drop=True)
```

Why Remove Duplicates?

- Prevents distortion and bias
- if a large/small person is accounted like 10x, it will skew the average

- Save storage, avoid multiple responses

Exploratory Plots



Histogram of height and weight
- the average (mean)
- the median (center point of all samples sorted)

Exploratory Plots

```
import seaborn as sns

# --- Height Histogram ---
mean_h = df_clean["Height"].mean()
median_h = df_clean["Height"].median()

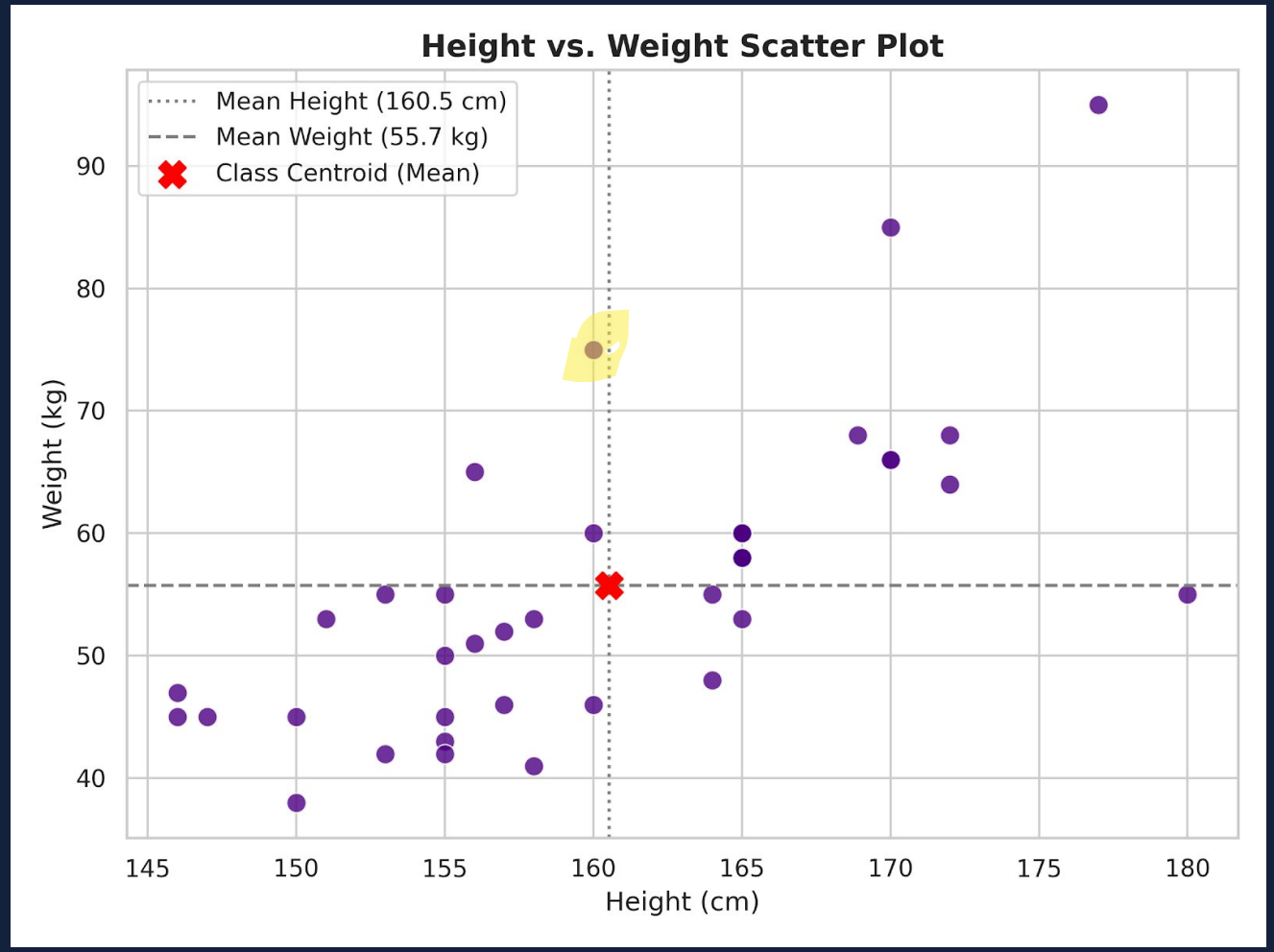
sns.histplot(df_clean["Height"], kde=True, ax=axes[0], color="skyblue",
bins=8)
axes[0].axvline(
    mean_h,
    color="red",
    linestyle="--",
    linewidth=2,
    label=f"Mean: {mean_h:.1f} cm",
)
axes[0].axvline(
    median_h,
    color="green",
    linestyle="-",
    linewidth=2,
    label=f"Median: {median_h:.1f} cm",
)
axes[0].set_title("Height Distribution", fontsize=14, fontweight="bold")
axes[0].set_xlabel("Height (cm)")
axes[0].set_ylabel("Student Count")
axes[0].legend()
```

```
# --- Weight Histogram ---
mean_w = df_clean["Weight"].mean()
median_w = df_clean["Weight"].median()

sns.histplot(df_clean["Weight"], kde=True, ax=axes[1], color="salmon", bins=8)
axes[1].axvline(
    mean_w,
    color="red",
    linestyle="--",
    linewidth=2,
    label=f"Mean: {mean_w:.1f} kg",
)
axes[1].axvline(
    median_w,
    color="green",
    linestyle="-",
    linewidth=2,
    label=f"Median: {median_w:.1f} kg",
)
axes[1].set_title("Weight Distribution", fontsize=14, fontweight="bold")
axes[1].set_xlabel("Weight (kg)")
axes[1].set_ylabel("Student Count")
axes[1].legend()

plt.tight_layout()
plt.show()
```

Exploratory Plots



Centroid (center of the elements, on the two dimensions)

Exploratory Plots

```
plt.figure(figsize=(8, 6))

# Plot data points
sns.scatterplot(
    data=df_clean, x="Height", y="Weight", color="indigo", s=80, alpha=0.8
)

# Reference lines for means
plt.axvline(
    mean_h, color="gray", linestyle=":", label=f"Mean Height ({mean_h:.1f} cm)"
)
plt.axhline(
    mean_w, color="gray", linestyle="--", label=f"Mean Weight ({mean_w:.1f} kg)"
)

# Plot class centroid point
plt.scatter(
    [mean_h],
    [mean_w],
    color="red",
    s=150,
    zorder=5,
    marker="X",
    label="Class Centroid",
)

plt.title("Height vs. Weight Scatter Plot", fontsize=14, fontweight="bold")
plt.xlabel("Height (cm)")
plt.ylabel("Weight (kg)")
plt.legend()
plt.tight_layout()
```

Derived Data: BMI (Body Mass Index)



WEIGHT lbs	100	105	110	115	120	125	130	135	140	145	150	155	160	165	170	175	180	185	190	195	200	205	210	215
kgs	45.5	47.7	50.0	52.3	54.5	56.8	59.1	61.4	63.6	65.9	68.2	70.5	72.7	75.0	77.3	79.5	81.8	84.1	86.4	88.6	90.9	93.2	95.5	97.7
HEIGHT in/cm																								
5'0" - 152.4	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42
5'1" - 154.9	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	36	37	38	39	40
5'2" - 157.4	18	19	20	21	22	23	23	24	25	26	27	28	29	30	31	32	33	33	34	35	36	37	38	39
5'3" - 160.0	17	18	19	20	21	22	23	24	24	25	26	27	28	29	30	31	32	32	33	34	35	36	37	38
5'4" - 162.5	17	18	18	19	20	21	22	23	24	24	25	26	27	28	29	30	31	31	32	33	34	35	36	37
5'5" - 165.1	16	17	18	19	20	20	21	22	23	24	25	25	26	27	28	29	30	30	31	32	33	34	35	35
5'6" - 167.6	16	17	17	18	19	20	21	21	22	23	24	25	25	26	27	28	29	29	30	31	32	33	34	34
5'7" - 170.1	15	16	17	18	18	19	20	21	22	22	23	24	25	25	26	27	28	29	29	30	31	32	33	33
5'8" - 172.7	15	16	16	17	18	19	19	20	21	22	22	23	24	25	25	26	27	28	28	29	30	31	32	32
5'9" - 175.2	14	15	16	17	17	18	19	20	20	21	22	22	23	24	25	25	26	27	28	28	29	30	31	31
5'10" - 177.8	14	15	15	16	17	18	18	19	20	20	21	22	23	23	24	25	25	26	27	28	28	29	30	30
5'11" - 180.3	14	14	15	16	16	17	18	18	19	20	21	21	22	23	23	24	25	25	26	27	28	28	29	30
6'0" - 182.8	13	14	14	15	16	17	17	18	19	19	20	21	21	22	23	23	24	25	25	26	27	27	28	29
6'1" - 185.4	13	13	14	15	15	16	17	17	18	19	19	20	21	21	22	23	23	24	25	25	26	27	27	28
6'2" - 187.9	12	13	14	14	15	16	16	17	18	18	19	19	20	21	21	22	23	23	24	25	25	26	27	27
6'3" - 190.5	12	13	13	14	15	15	16	16	17	18	18	19	20	20	21	21	22	23	23	24	25	25	26	26
6'4" - 193.0	12	12	13	14	14	15	15	16	17	17	18	18	19	20	20	21	22	22	23	23	24	25	25	26

https://www.hdfclife.com/financial-tools-calculators/bmi-calculator

Adolphe Quetelet (1796–1874)—the average man and indices of obesity
https://academic.oup.com/ndt/article-abstract/23/1/47/1923176

Derived Data: BMI (Body Mass Index)

$$BMI = \frac{weight}{height^2}$$

Weight in KG
Height in meter

0	26083010017@student.upnjatim.ac.id	155.0	55	0.002289	Underweight
1	26083010018@student.upnjatim.ac.id	165.0	53	0.001947	Underweight
2	26083010019@student.upnjatim.ac.id	172.0	64	0.002163	Underweight
3	26083010036@student.upnjatim.ac.id	180.0	55	0.001698	Underweight
4	26083010037@student.upnjatim.ac.id	155.0	45	0.001873	Underweight
5	26083010038@student.upnjatim.ac.id	165.0	60	0.002204	Underweight
6	26083010039@student.upnjatim.ac.id	177.0	95	0.003032	Underweight
7	26083010056@student.upnjatim.ac.id	151.0	53	0.002324	Underweight
8	26083010057@student.upnjatim.ac.id	170.0	85	0.002941	Underweight

Derived Data: BMI (Body Mass Index)

```
df_clean['BMI'] = df_clean['Weight'] / (df_clean['Height'] ** 2)

# Categorize BMI according to WHO standard categories
def get_bmi_category(bmi):
    if bmi < 18.5:
        return 'Underweight'
    elif 18.5 <= bmi < 25.0:
        return 'Normal weight'
    elif 25.0 <= bmi < 30.0:
        return 'Overweight'
    else:
        return 'Obese'

df_clean['Category'] = df_clean['BMI'].apply(get_bmi_category)

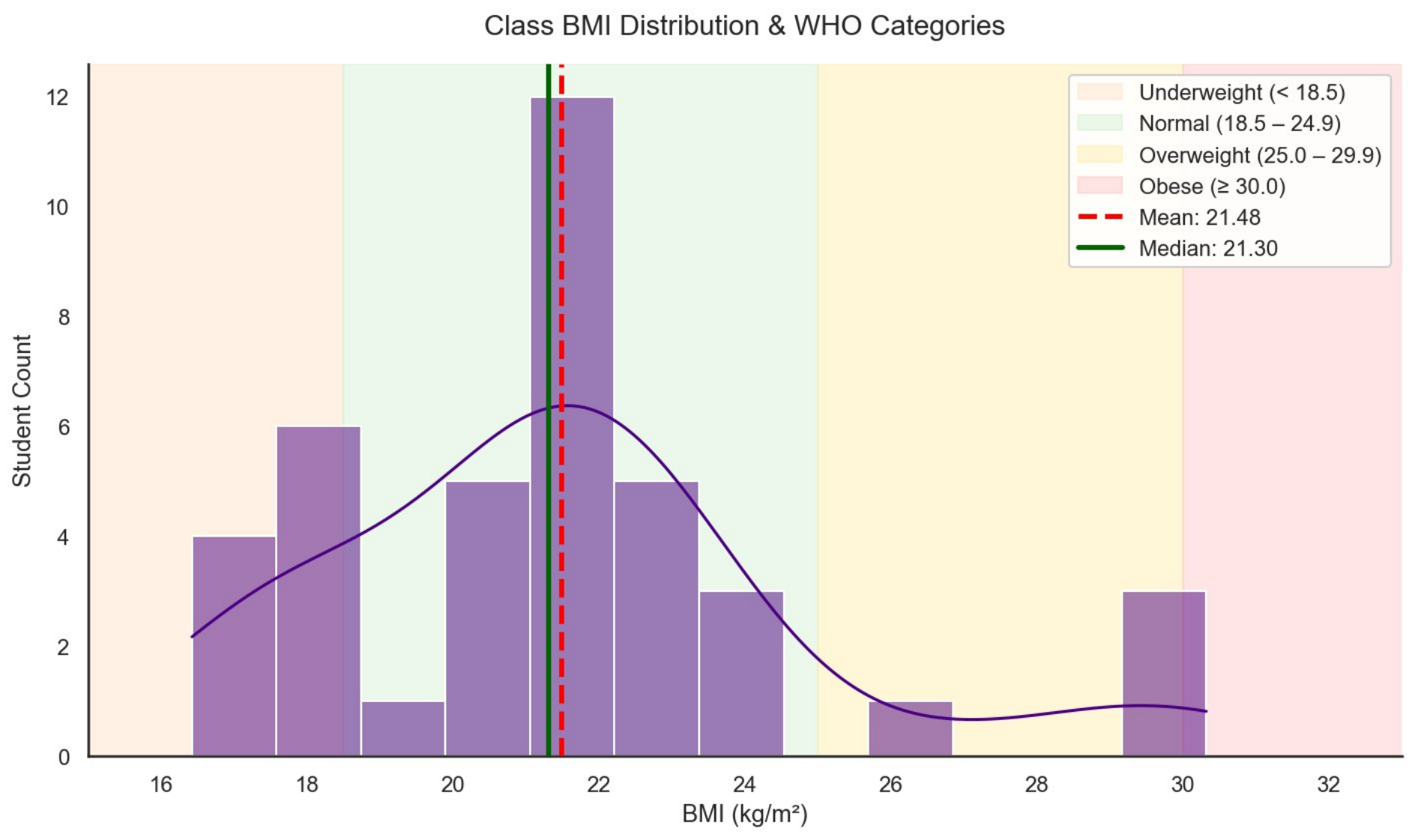
print("BMI Summary Statistics:")
print(df_clean['BMI'].describe())

print("\nCategory Distribution:")
print(df_clean['Category'].value_counts())

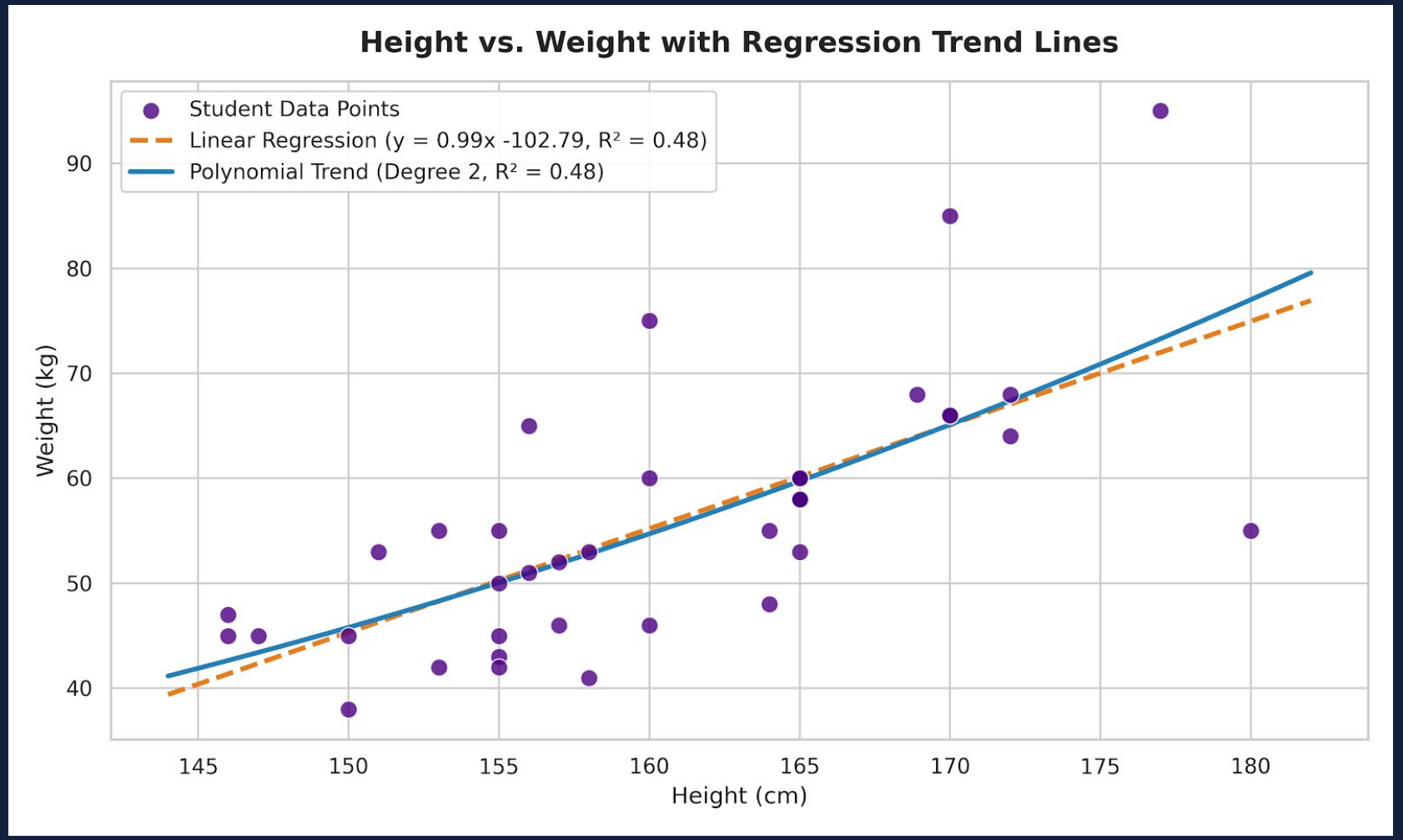
print("\nFirst 10 records with BMI:")
print(df_clean[['Email Address', 'Height', 'Weight', 'BMI', 'Category']].head(10))
```

https://www.who.int/data/gho/data/themes/topics/topic-details/GHO/body-mass-index

Derived Data: BMI (Body Mass Index)



Trends



Taller = heavier

Trends

```
# Calculate Linear Fit (Degree 1)
z_linear = np.polyfit(height, weight, 1)
p_linear = np.poly1d(z_linear)

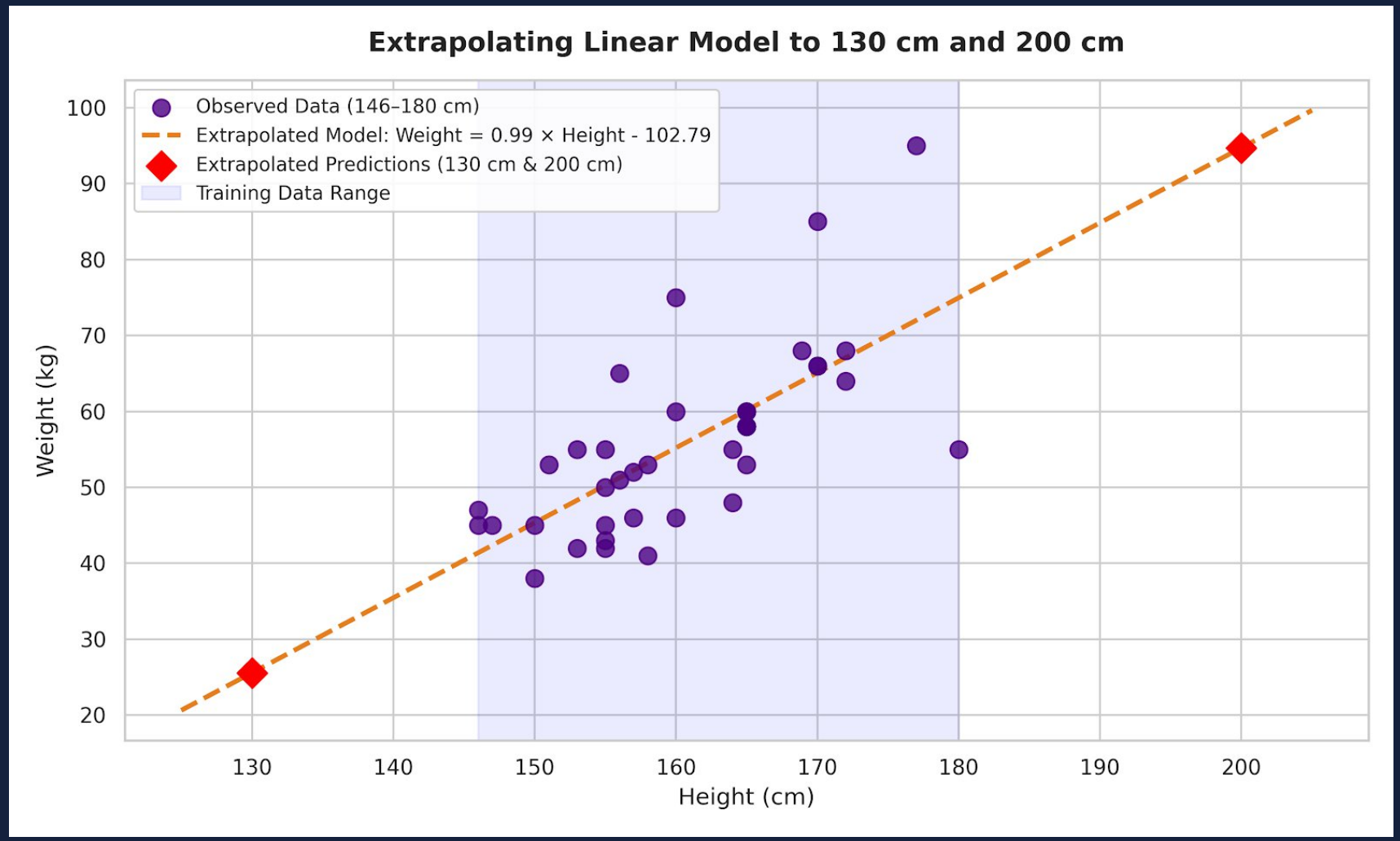
# Calculate Polynomial Fit (Degree 2)
z_poly = np.polyfit(height, weight, 2)
p_poly = np.poly1d(z_poly)

# X range for plotting smooth lines
x_range = np.linspace(height.min() - 2, height.max() + 2, 100)

# Calculate R-squared for Linear Fit
y_pred_lin = p_linear(height)
r2_lin = 1 - (
    np.sum((weight - y_pred_lin) ** 2) / np.sum((weight - weight.mean()) ** 2)
)

# Calculate R-squared for Polynomial Fit
y_pred_poly = p_poly(height)
r2_poly = 1 - (
    np.sum((weight - y_pred_poly) ** 2) / np.sum((weight - weight.mean()) ** 2)
)
```

Extrapolating/Predicting



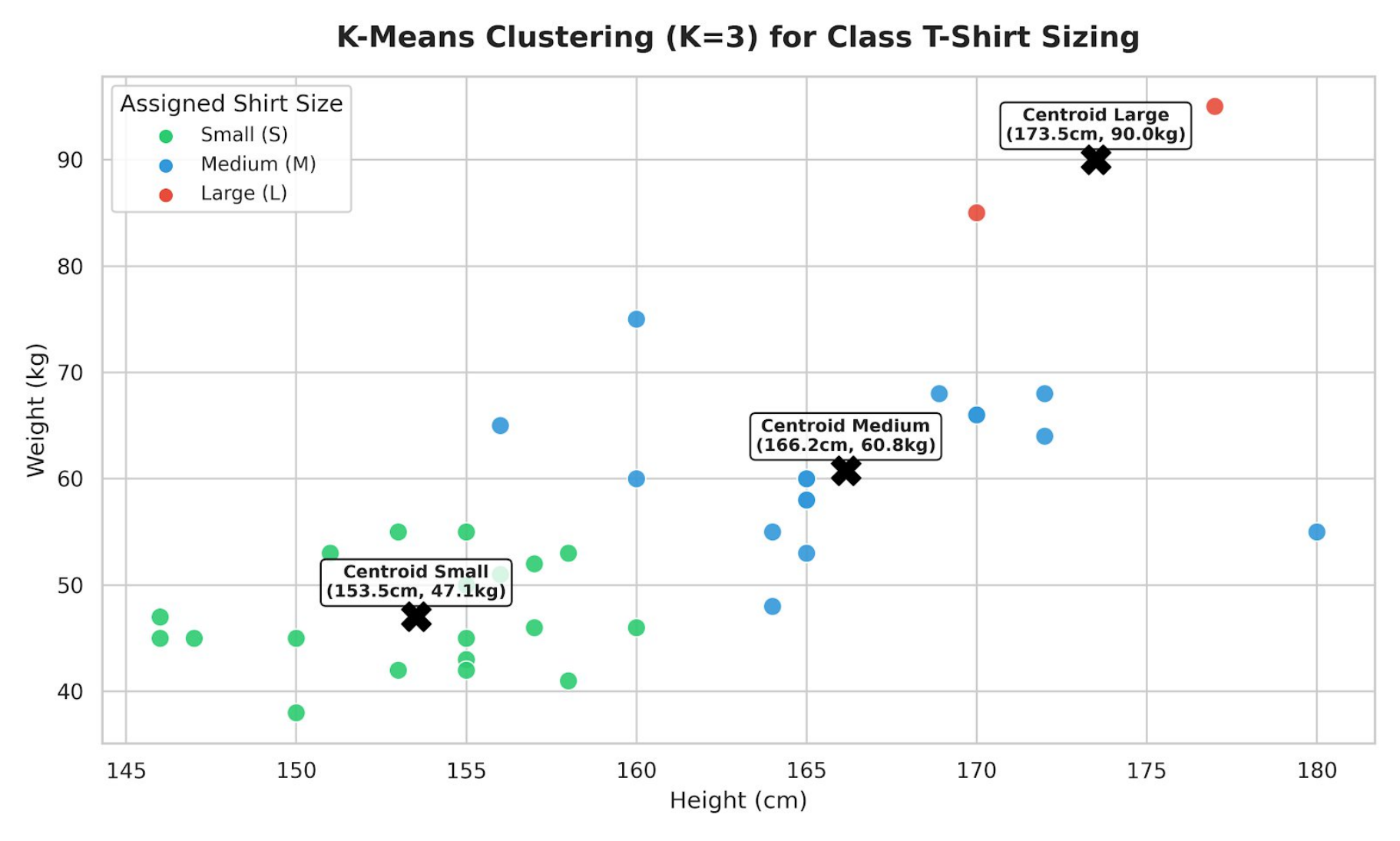
Weight (kg) = 0.9875 * Height (cm) - 102.79

Extraploation

```
extrap_heights = [130, 140, 150, 160, 170, 180, 190, 200]

print("Linear Regression Formula:")
print(f"Weight (kg) = {z_linear[0]:.4f} * Height (cm) {z_linear[1]:+.4f}")
print("\nExtrapolated Estimates:")
for h in extrap_heights:
    w_pred = p_linear(h)
    bmi_pred = w_pred / ((h / 100) ** 2)
    print(f"Height: {h} cm -> Predicted Weight: {w_pred:.2f} kg (Predicted BMI:
{bmi_pred:.2f})")
```

Shirt Sizes, again



K-means clustering

```
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

height = df_clean['Height'].values
weight = df_clean['Weight'].values

# Combine features into matrix X
X = np.column_stack((height, weight))

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 2. Fit K-Means with K=3 (Small, Medium, Large)
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
labels = kmeans.fit_predict(X_scaled)
centroids_scaled = kmeans.cluster_centers_
centroids_unscaled = scaler.inverse_transform(centroids_scaled)

# Assign cluster labels based on height/weight centroids (S: lowest height/weight, L: highest)
cluster_order = np.argsort(centroids_unscaled[:, 0])  # Sort by height centroid
size_names = ['Small (S)', 'Medium (M)', 'Large (L)']
cluster_mapping = {original_idx: size_names[new_rank] for new_rank, original_idx in enumerate(cluster_order)}

df_clean['Cluster_ID'] = labels
df_clean['Shirt_Size'] = df_clean['Cluster_ID'].map(cluster_mapping)

# Print Summary Breakdown
size_counts = df_clean['Shirt_Size'].value_counts().reindex(['Small (S)', 'Medium (M)', 'Large (L)'])
print("--- Shirt Size Inventory Order ---")
print(size_counts)
```

Recap

- Cleaning (removing duplicate)
- Statistical average (mean, median)
- Histogram
- Derivative metric
- Trend lines (regression)
- Prediction via extrapolation
- Clustering

Assignment

Rerun the code on python notebook, make the necessary adjustments https://github.com/jendralhxr/introdatascience/tree/main/beratbadan (plain python code and data here)

In each plot, mark/draw yourself with **a star** (or any marking point)

For the shirt sizes classification, change to **4 or 5 groups,** use BMI

Submission due: 2026-11-16 noon, late submission will have grade deducted (5 point /6 hours)

Send via email to muhammad_zulhaj.sada@upnjatim.ac.id